1. qbp-sim User Guide

`qbp-sim` runs continuous-time Gillespie simulations for quantum-network routing experiments based on backpressure-style control. The main interfaces are typed JSON configs, the `qbp-sim` CLI, and the Python facade.

The simulator records concrete event traces. A trace can be replayed to reconstruct the same state transitions without resampling randomness.

The `qbp_sim.lp` module solves a global average-rate linear program. It produces LP outputs and simulator configs for benchmark instances. The LP is an aggregate-rate optimizer; the Gillespie simulator is a stochastic event process.

2. Installation

Use `uv`:

```
uv sync --all-groups
uv run qbp-sim --help
```

Build this manual with Typst:

```
typst compile docs/manual.typ docs/qbp-sim.pdf
```

3. Quick Start

Run the built-in four-node example:

```
uv run qbp-sim example --until 50 --seed 0
```

Run a JSON config:

```
uv run qbp-sim run --config docs/examples/basic_config.json --until 50
```

Write a Vortex event trace:

```
uv run qbp-sim run \
  --config docs/examples/basic_config.json \
  --until 50 \
  --trace output/traces/basic.vortex
```

Write a Parquet event trace:

```
uv run qbp-sim run \
  --config docs/examples/basic_config.json \
  --until 50 \
  --trace output/traces/basic.parquet \
  --trace-format parquet
```

Replay a trace:

```
uv run qbp-sim replay --trace output/traces/basic.vortex --until 50
```

Write snapshots and plot service ratio:

```
uv run qbp-sim run \
  --config docs/examples/basic_config.json \
  --until 50 \
  --snapshots output/snapshots/basic.jsonl.zst

uv run qbp-sim analyze \
  --snapshots output/snapshots/basic.jsonl.zst \
  --plot-metric service_ratio \
  --plot-out output/plots/basic_service_ratio.png
```

4. Simulation Configs

A simulation config describes an undirected network. Matrices are symmetric. Node ids are matrix row and column indexes.

- `generation_rates[x][y]`: Bell-pair generation rate on edge `(x, y)`.
- `consumption_rates[x][y]`: demand arrival rate for pair `(x, y)`.
- `swap_rates[i]`: swap opportunity rate at node `i`.
- `capacity_headroom`: multiplier on generation, swap, and service opportunity rates. Demand arrivals are unchanged. The default is `1.01`.
- `virtual_swap_policy`: swap-selection policy.
- `instant_service_fulfillment`: deterministic physical service realization after inventory-producing events.
- `instant_swap_fulfillment`: deterministic physical swap realization through the local frontier.

Minimal config:

```
{
  "generation_rates": [
    [0.0, 1.0, 1.0, 0.0],
    [1.0, 0.0, 0.0, 1.0],
    [1.0, 0.0, 0.0, 1.0],
    [0.0, 1.0, 1.0, 0.0]
  ],
  "consumption_rates": [
    [0.0, 0.0, 0.0, 2.0],
    [0.0, 0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0, 0.0],
    [2.0, 0.0, 0.0, 0.0]
  ],
  "swap_rates": [0.0, 1.0, 1.0, 0.0],
  "capacity_headroom": 1.01,
  "virtual_swap_policy": {"mode": "bp"}
}
```

Supported policy modes:

- `bp`: full-information backpressure. Each node scans all swap candidates and chooses the largest positive virtual pressure.
- `limited_info_bp`: backpressure with `k` queried candidates and `memory` remembered candidates per node.
- `max_min`: path-oblivious inventory balancing. It uses physical inventory `Q` rather than scarcity `alpha`.
- `limited_info_max_min`: max-min with `k` queried candidates and `memory` remembered candidates per node.

Limited-information policy config:

```
{
  "virtual_swap_policy": {
    "mode": "limited_info_bp",
    "k": 4,
    "memory": 8
  }
}
```

5. State And Events

The simulator stores dense symmetric state matrices:

- `Q[x, y]`: Bell-pair inventory.
- `D[x, y]`: virtual demand backlog, also called `gamma`.
- `alpha[x, y]`: backpressure scarcity signal.
- $H^{R[x,y]}$: pending physical service requests.
- $H^{\mu[i,y,z]}$: pending physical swap requests.

The Gillespie engine samples these stochastic event families:

- demand arrivals
- Bell-pair generation
- virtual service requests
- direct service admission for max-min mode
- virtual swap requests

- max-min physical swaps
- physical service realizations
- physical swap realizations

Backpressure virtual events update `D`, `alpha`, H^R, and H^μ. Physical realization events consume inventory from `Q`.

The deterministic frontier runs after sampled events when instant fulfillment is enabled. It realizes pending service before pending swaps on the affected edge. Deterministic realization events use the same event trace schema as sampled events and have zero elapsed time.

6. Running Simulations

Run a user-provided config:

```
uv run qbp-sim run --config CONFIG.json --until 100 --seed 1
```

Common options:

- `--max-events 0`: no event cap. Positive values stop after that many sampled events.
- `--sample-every N`: record aggregate snapshots every `N` applied events.
- `--trace OUTFILE`: write every event.
- `--trace-format vortex|parquet|jsonl_zst`: select the trace writer.
- `--trace-float-precision float16|float32|float64`: precision for columnar trace floats.
- `--trace-time-mode full|none`: include or omit timing and rate fields.
- `--snapshots OUTFILE`: write sampled aggregate snapshots.
- `--virtual-swap-policy MODE`: override the config policy.
- `--swap-k K`: limited-information query count.
- `--swap-memory M`: limited-information memory size.
- `--instant-service-fulfillment`: enable deterministic immediate service realization.
- `--instant-swap-fulfillment`: enable deterministic immediate swap realization.

7. Traces, Snapshots, And Metadata

Traces are per-event logs for replay and detailed analysis. Supported formats:

- `.vortex`
- `.parquet`
- `.jsonl.zst`

Columnar traces store `time`, `total_rate`, and `event_rate` as `float32` by default. `--trace-float-precision float64` stores full precision. `--trace-time-mode none` omits time and rate fields. Timeless traces preserve event order and state transitions.

Snapshots are sampled aggregate summaries. They are useful for quick plots and summaries.

Experiment runs write:

- `lp_solution.json`
- `simulation_config.json`
- `events.<format>`
- `run_metadata.json`

Matrix runs also write `summary.csv`.

8. Metrics

The primary service metric is:

```
service_ratio = services_completed / demand_arrivals
```

Common counters:

- `total_backlog`: aggregate pending virtual or physical work.
- `total_inventory`: Bell pairs stored in `Q`.

- `total_scarcity`: aggregate backpressure scarcity `alpha`.
- `pair_generations`: generated Bell pairs.
- `virtual_service_requests`: virtual service requests.
- `virtual_swap_requests`: virtual swap requests.
- `services_completed`: fulfilled demand requests.
- `swaps_completed`: realized physical swaps.

A CLI run prints:

```
QBP Gillespie simulation
time=50.000000
events=292
backlog=14
inventory=9
scarcity=5
demand_arrivals=87
pair_generations=101
virtual_service_requests=80
virtual_swap_requests=24
services_completed=73
swaps_completed=22
```

A `run_metadata.json` file stores reproducibility data and final counters:

```
{
  "schema_version": 1,
  "command": "matrix",
  "n_nodes": 3,
  "seed": 3,
  "until_time": 2.0,
  "trace_format": "vortex",
  "trace_time_mode": "none",
  "simulation_config_path": ".../simulation_config.json",
  "trace_path": ".../events.vortex",
  "result": {
    "final_time": 2.0,
    "events_processed": 8,
    "demand_arrivals": 1,
    "services_completed": 1,
    "service_ratio": 1.0
  }
}
```

9. Experiment Matrices

An experiment matrix expands Cartesian products of topology, graph size, demand sparsity, headroom, policy, rate scales, seeds, trace settings, and instant-fulfillment modes.

```
{
  "topologies": ["cycle"],
  "graph_sizes": [3],
  "consumption_edge_fractions": [null],
  "headrooms": [1.01],
  "policies": [
    {"mode": "bp", "label": "bp"},
    {"mode": "limited_info_bp", "k": 1, "memory": 1, "label": "limited BP k=1, m=1"}
  ],
  "edge_weights": [10.0],
  "gen_scales": [10.0],
  "cons_scales": [0.2],
  "swap_rates": [20.0],
  "seed_offsets": [0],
  "until_time": 2.0,
  "sample_every": 10,
  "trace_format": "vortex",
```

```
  "trace_float_precision": "float32",
  "trace_time_mode": "none"
}
```

Run a matrix:

```
uv run qbp-sim matrix \
  --config docs/examples/matrix_config.json \
  --output-dir output/matrix_demo
```

Inspect expanded cases:

```
uv run qbp-sim matrix \
  --config docs/examples/matrix_config.json \
  --output-dir output/matrix_demo \
  --dry-run
```

10. Plots

Built-in plots use Altair/Vega-Lite. Output format follows the file extension: `.png`, `.svg`, `.html`, or `.json`.

10.1. Snapshot Metric Plot

Plot one snapshot metric over time:

```
uv run qbp-sim run \
  --config docs/examples/basic_config.json \
  --until 50 \
  --sample-every 25 \
  --snapshots output/snapshots/basic.jsonl.zst

uv run qbp-sim analyze \
  --snapshots output/snapshots/basic.jsonl.zst \
  --plot-metric service_ratio \
  --plot-out output/plots/basic_service_ratio.png
```

Available snapshot metrics include `event_index`, `time`, `total_backlog`, `total_inventory`, `total_scarcity`, `demand_arrivals`, `pair_generations`, `services_completed`, `swaps_completed`, and `service_ratio`.

10.2. Snapshot Metric Series Plot

Compare one snapshot metric across several runs from Python:

```
from qbp_sim.analysis import plot_snapshot_metric_series
from qbp_sim.snapshots import SnapshotReader

def load(path):
    with SnapshotReader(path) as reader:
        return list(reader)

plot_snapshot_metric_series(
    [
        ("bp", load("output/bp/snapshots.jsonl.zst")),
        ("limited", load("output/limited/snapshots.jsonl.zst")),
    ],
    "service_ratio",
    "output/plots/service_ratio_series.html",
    title="service ratio comparison",
)
```

10.3. Cycle Service-Gap Plot

Run LP-derived cycle instances and plot `1 - service_ratio` on log-log axes:

```
uv run qbp-sim cycle-service-ratio \
  --sizes 4 8 16 \
  --until 1000 \
  --plot-out output/plots/cycle_service_ratio_gap.png
```

10.4. Headroom Service-Gap Plot

Compare capacity headroom values on one LP-derived cycle instance:

```
uv run qbp-sim headroom-service-ratio \
  --n 16 \
  --headrooms 1.0 1.01 1.05 \
  --until 1000 \
  --plot-out output/plots/headroom_service_ratio_gap.svg
```

10.5. Limited-Information Service-Ratio Plot

Compare full-information backpressure with `limited_info_bp` policies:

```
uv run qbp-sim limited-info-service-ratio \
  --n 8 \
  --limited-policies 1:1 2:2 4:4 \
  --until 1000 \
  --plot-start-time 10 \
  --plot-out output/plots/limited_info_service_ratio.png
```

10.6. LP Benchmark Ratio Plot

Compare path-based routing with the LP benchmark on random torus-grid instances:

```
from qbp_sim.lp.linear import plot_swaps_and_maxgen_vs_nodes

plot_swaps_and_maxgen_vs_nodes(
    grid_sizes=[2, 3, 4],
    trials=3,
    save_path="output/plots/lp_path_ratios.png",
    seed=0,
)
```

11. Public Python API

Import the stable user API from `qbp_sim`:

```
from qbp_sim import RunOptions, TraceFormat, build_four_node_example_config, run_simulation

config = build_four_node_example_config()
output = run_simulation(
    config,
    RunOptions(
        until_time=50.0,
        seed=0,
        trace_path="output/traces/python_example.parquet",
        trace_format=TraceFormat.PARQUET,
    ),
)
print(output.service_ratio)
```

Public root types:

- `SimulationInputConfig`
- `VirtualSwapPolicyConfig`
- `ExperimentMatrixConfig`
- `ExperimentPolicyConfig`
- `ExperimentMatrixCase`
- `RunOptions`
- `RunOutput`
- `VirtualSwapPolicyMode`
- `TopologyName`
- `TraceFloatPrecision`
- `TraceFormat`
- `TraceTimeMode`

Public root functions:

- `load_simulation_config(path)`
- `load_experiment_matrix_config(path)`
- `build_four_node_example_config()`
- `run_simulation(config_or_path, options=None)`
- `replay_trace(trace_path, config=None, final_time=None, sample_every=500, snapshots_path=None)`

11.1. Replay From Python

```
from qbp_sim import RunOptions, load_simulation_config, replay_trace, run_simulation

config = load_simulation_config("docs/examples/basic_config.json")
run = run_simulation(
    config,
    RunOptions(
        until_time=50.0,
        trace_path="output/traces/python_example.vortex",
    ),
)
replayed = replay_trace(
    "output/traces/python_example.vortex",
    config=config,
    final_time=run.result.final_time,
)
assert replayed.result.services_completed == run.result.services_completed
```

11.2. Parameter Sweep From Python

```
from qbp_sim import RunOptions, VirtualSwapPolicyMode, build_four_node_example_config,
run_simulation
from qbp_sim.config import VirtualSwapPolicyConfig

base = build_four_node_example_config()

for policy in [VirtualSwapPolicyMode.BP, VirtualSwapPolicyMode.MAX_MIN]:
    config = base.model_copy(
        update={"virtual_swap_policy": VirtualSwapPolicyConfig(mode=policy)}
    )
    output = run_simulation(
        config,
        RunOptions(until_time=100.0, seed=0, sample_every=100),
    )
    print(policy, output.service_ratio)
```

11.3. Matrix Expansion From Python

```
from qbp_sim import load_experiment_matrix_config

matrix = load_experiment_matrix_config("docs/examples/matrix_config.json")
for case in matrix.cases():
    print(case.slug, case.policy_label, case.seed)
```

11.4. Trace Analysis With Polars

```
import polars as pl
import vortex as vx

lf = vx.open("output/traces/python_example.vortex").to_polars()
# For Parquet:
lf = pl.scan_parquet("output/traces/python_example.parquet")

service_ratio = (
    lf.select("event_index", "time", "event_type")
    .with_columns(
        demand_arrivals=(pl.col("event_type") == "demand_arrival").cast(pl.Int64).cum_sum(),
```

```
        services_completed=(pl.col("event_type") == "physical_service").cast(pl.Int64).cum_sum(),
    )
    .with_columns(
        service_ratio=pl.when(pl.col("demand_arrivals") > 0)
        .then(pl.col("services_completed") / pl.col("demand_arrivals"))
        .otherwise(0.0)
    )
    .select("event_index", "time", "service_ratio")
    .collect()
)
```

12. LP Benchmark

The LP module solves global average-rate optimization problems and can emit simulator configs.

```
from qbp_sim.lp import linear

num_nodes = 6
generation_capacity = linear.create_cycle_adjacency_matrix(num_nodes, edge_weight=10.0)
consumption_demand = linear.create_sparse_symmetric_adjacency_matrix(
    num_nodes,
    edge_fraction=0.3,
    max_edge_weight=7.0,
    seed=0,
    min_positive_edges=1,
)

spec = linear.LinearSpec(num_nodes)
spec.add_generate_constraints(generation_capacity)
spec.add_consume_constraints(consumption_demand)
spec.add_swap_capacity_constraints([20.0] * num_nodes)
lp_result = spec.solve(objective="min_sum_generate")
config = linear.build_lp_solution_simulation_input_config(spec=spec, lp_result=lp_result)
```

13. Package Layout

- `src/qbp_sim/facade.py`: public Python API.
- `src/qbp_sim/core/`: simulator state, kernels, event producer, event applier, frontier realization, run loop, and replay.
- `src/qbp_sim/io/`: event records, event traces, and snapshots.
- `src/qbp_sim/config/`: Pydantic config models and runtime conversion.
- `src/qbp_sim/analysis/`: snapshot summaries and Altair chart helpers.
- `src/qbp_sim/experiments/`: LP-derived experiment setup, metadata, matrix execution, and plots.
- `src/qbp_sim/cli/`: command-line parser and command handlers.
- `src/qbp_sim/lp/`: LP benchmark model.
- `examples/`: runnable Python examples.
- `docs/`: Typst manual and JSON examples.
- `tests/`: unit, replay, trace, experiment, LP, import-contract, and gated stochastic tests.

14. Performance And Reproducibility

- Use `pueue` for long simulations, benchmark jobs, and analysis runs.
- Use `.vortex` for compact columnar event logs.
- Use `trace_format: "parquet"` for standard Parquet tooling.
- Use fixed seeds for comparable runs.
- Keep `run_metadata.json` with every run.
- Keep generated runs under ignored `output/`.

15. Building Artifacts

Build a source distribution and wheel:

```
uv build
```

The build writes:

```
dist/qbp_sim-0.2.0.tar.gz
dist/qbp_sim-0.2.0-py3-none-any.whl
```

The GitHub Actions `build` workflow runs tests, compiles this manual, builds the wheel and source distribution, and uploads those files as workflow artifacts.

16. Glossary

- `Q`: physical Bell-pair inventory matrix.
- `alpha`: backpressure scarcity signal.
- virtual demand backlog: demand accumulated before admission to physical service.
- service backlog, H^R: pending service requests waiting for physical inventory.
- swap deficit, H^μ: pending virtual swaps waiting for physical realization.
- headroom: multiplier applied to generation, swap, and service opportunities.
- trace: per-event log for replay and analysis.
- snapshot: sampled aggregate state summary.
- service ratio: fulfilled requests divided by demand arrivals.